

Evidencias de Entrega: API de Categorías con FastAPI

Nombre del Aprendiz

Jorge Alejandro Torres Paez

Servicio Nacional de Aprendizaje (SENA)

Tecnólogo en Análisis y Desarrollo de Software (ADSO)

Ficha N.º 3407180

Tercer Trimestre — Pruebas de Software, Módulo III

Instructor(a)

Jeison Aly Contreraz

Fecha de Entrega

13/09/2026

Evidencias de Entrega: API de Categorías con FastAPI

Propósito del Documento

Este documento lo elaboré para organizar las evidencias que se piden en la actividad autónoma de la API de Categorías con FastAPI, del Módulo III de Pruebas de Software. En cada punto explico brevemente qué debe mostrar la imagen y dejo el espacio para pegar la captura de pantalla correspondiente, con el fin de dejar constancia del trabajo realizado durante la actividad.

Evidencias Fotográficas y Capturas de Pantalla

Captura de Swagger (/docs)

Descripción. En esta parte pego la captura de la interfaz Swagger, que es la documentación que genera FastAPI de forma automática en la ruta /docs. Ahí se pueden ver todos los endpoints que desarrollé para la API de categorías.

Elementos que debe evidenciar la imagen:

- Se ve la URL /docs abierta en el navegador.
- Aparecen los seis endpoints que pide la actividad: GET /categories, GET /categories/{category_id}, POST /categories, PATCH /categories/{category_id}, DELETE /categories/{category_id} y el filtro ?active=.
- Se alcanzan a leer los métodos HTTP de cada endpoint (GET, POST, PATCH, DELETE).

Captura de imagen:

Funcionamiento		^
GET	/ Root	▼
GET	/health Health Check	▼
Products		^
GET	/products/ Get Product	▼
GET	/products/{product_id} Get Product By Id	▼
PATCH	/products/{product_id} Update Product	▼
DELETE	/products/{product_id} Delete Product	▼
POST	/products Create Product	▼
Categories		^
GET	/categories Get Categories	▼
POST	/categories Create Category	▼
GET	/categories/{category_id} Get Category By Id	▼
PATCH	/categories/{category_id} Update Category	▼
DELETE	/categories/{category_id} Delete Category	▼
Schemas		^

Captura de Ejecución de pytest -v

Descripción. En esta evidencia pego la captura de la terminal donde se ve la ejecución completa de las pruebas automatizadas que hice con pytest, usando el comando `pytest -v`.

Elementos que debe evidenciar la imagen:

- Se ve el comando que ejecuté (`pytest -v`).
- Se alcanzan a leer los nombres de las pruebas que corrí (mínimo diez; en mi caso hice las doce de la matriz CA01 a CA12).
- Se ve el resultado final de la ejecución (PASSED o FAILED) y el resumen con la cantidad de pruebas.

Captura de imagen:

```

(.venv) PS C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias> pytest --
===== test session starts =====
platform win32 -- Python 3.10.7, pytest 8.1.1, pluggy 1.6.0 -- C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias
plugins: anyio-4.10.1
collected 23 items

test/test_categories.py::test_list_categories PASSED [ 4%]
test/test_categories.py::test_get_existing_category PASSED [ 8%]
test/test_categories.py::test_get_non_existing_category PASSED [ 13%]
test/test_categories.py::test_get_invalid_id PASSED [ 17%]
test/test_categories.py::test_create_valid_category PASSED [ 21%]
test/test_categories.py::test_create_category_name_too_short PASSED [ 26%]
test/test_categories.py::test_create_category_missing_name PASSED [ 30%]
test/test_categories.py::test_update_existing_category PASSED [ 34%]
test/test_categories.py::test_update_non_existing_category PASSED [ 39%]
test/test_categories.py::test_delete_existing_category PASSED [ 43%]
test/test_categories.py::test_delete_non_existing_category PASSED [ 47%]
test/test_categories.py::test_filter_active_categories PASSED [ 52%]
test/test_categories.py::test_search_categories PASSED [ 56%]
test/test_categories.py::test_search_categories_no_match PASSED [ 60%]
test/test_products.py::test_health PASSED [ 65%]
test/test_products.py::test_get_products PASSED [ 69%]
test/test_products.py::test_get_existing_product PASSED [ 73%]
test/test_products.py::test_get_non_existing_product PASSED [ 78%]
test/test_products.py::test_create_product PASSED [ 82%]
test/test_products.py::test_update_product PASSED [ 86%]
test/test_products.py::test_update_product_stock_recalculates_available PASSED [ 91%]
test/test_products.py::test_delete_product PASSED [ 95%]
test/test_products.py::test_filters PASSED [100%]

===== warnings summary =====
.venv\Libsite-packages\starlette\testclient.py:53
  C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias\.venv\Libsite-packages\starlette\testclient.py:53: DeprecationWarning: The anyio.bac.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _portal_factory_type = Callable[[], AbstractContextManager[AnyioABCBlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 23 passed, 1 warning in 0.19s =====
(.venv) PS C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias>

```

Reto Opcional

Si alcancé a desarrollar la búsqueda por nombre (GET /categories?search=comp), en esta parte dejo la evidencia de que funciona correctamente, junto con las pruebas automatizadas que hice para esa funcionalidad.

Captura del Reto Opcional — Filtro de Búsqueda

Descripción. Pego aquí la captura de Swagger o de la terminal donde se ve que el parámetro de búsqueda por nombre funciona, y las pruebas automatizadas que hice para esta parte.

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories?search=... y la respuesta que devuelve.
- Se ven mínimo dos pruebas automatizadas relacionadas con esta funcionalidad.

Captura de imagen:

Categories

GET /categories Get Categories

Parameters

Cancel

Name	Description
active	--
boolean (boolean null) (query)	
search	Mice
string (string null) (query)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/categories?search=Mice' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/categories?search=Mice
```

Server response

Code	Details
200	Response body <pre>{ "name": "Mice", "description": "Ratón para computadora.", "active": false, "id": 7 }</pre> Response headers <pre>content-length: 88 content-type: application/json date: Sun, 13 Sep 2026 22:05:49 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Casos de Prueba (CA01–CA12)

(CA01- Listar categorías)

The screenshot displays a REST client interface with the following sections:

- GET /categories** (Method and Path)
- Parameters** (Query parameters):
 - active**: A dropdown menu with "--" selected. Type: `boolean | (boolean | null) (query)`.
 - search**: A text input field containing "search". Type: `string | (string | null) (query)`.
- Execute** (Button) and **Clear** (Button)
- Responses** (Section header)
- Curl**:

```
curl -X 'GET' \
  'http://127.0.0.1:8000/categories' \
  -H 'accept: application/json'
```
- Request URL**: `http://127.0.0.1:8000/categories`
- Server response**:
 - Code**: 200
 - Response body**:

```
{
  "name": "Laptops",
  "description": "Computadora portátil.",
  "active": true,
  "id": 1
},
{
  "name": "Smartphones",
  "description": "Teléfono celular inteligente.",
  "active": true,
  "id": 2
},
{
  "name": "Tablets",
  "description": "Tableta táctil.",
  "active": true,
  "id": 3
},
{
  "name": "Headphones",
  "description": "Aurífonos para música.",
  "active": true,
  "id": 4
},
{
  "name": "Monitors",
  "description": "Pantalla para computadora.",
  "active": true,
  "id": 5
}
```
 - Response headers**: (Empty)

(CA02- Consultar existente)

GET

/categories/{category_id} Get Category By Id

⌵

Parameters

Cancel

Name	Description
category_id * required	
integer	1
(path)	

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8000/categories/1' \n-M 'accept: application/json'

⌵

Request URL

http://127.0.0.1:8000/categories/1

⌵

Server response

Code	Details
200	Response body {\n "name": "Laptops",\n "description": "Computadora portátil.",\n "active": true,\n "id": 1\n} ⌵ Download Response headers content-length: 78\ncontent-type: application/json\ndate: Sun, 13 Sep 2026 22:13:39 GMT\nserver: uvicorn ⌵

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json Controls Accept header. Example Value Schema {\n "name": "Laptops",\n "description": "Computadora portátil.",\n "active": true,\n "id": 0\n}	
422	Validation Error	No links

(CA03- Consultar inexistente)

GET

/categories/{category_id} Get Category By Id

⌵

Parameters

Cancel

Name	Description
category_id * required	
integer	
(path)	
0	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/categories/0' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/categories/0
```

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

```
{
  "detail": "Category not found"
}
```

Download

Response headers

```
content-length: 31
content-type: application/json
date: Sun, 13 Sep 2026 22:16:24 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "Laptops", "description": "Computadora portátil.", "active": true, "id": 0 }</pre>		
422	Validation Error	No links

(CA04- ID inválido)

GET /categories/{category_id} Get Category By Id

Parameters

Cancel

Name	Description
category_id * required	abc
integer	
(path)	

Please correct the following validation errors and try again.

- For 'category_id': Value must be an integer.

Execute

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json		
Controls Accept header.		
Example Value Schema		
<pre>{ "name": "Laptops", "description": "Computadora portátil.", "active": true, "id": 0}</pre>		
422	Validation Error	No links

GET http://127.0.0.1:8000/categories/abc

Params Auth Headers Body Scripts

Key Value

Value

Status: 422 Unprocessable Content Size: 156 B Time: 4 ms

Response

```
1, {
2,   "detail": [
3,     {
4,       "type": "int_parsing",
5,       "loc": [
6,         "type",
7,         "category_id"
8,       ],
9,       "msg": "Input should be a valid integer, unable to parse string as an integer",
10      "type": "int"
11    }
12  ]
13}
```

Format Copy

Problems Output Debug Console Terminal Ports

```
[C:\Users\Usuario\Desktop\Proyectos_3ATP\test_fastapi_productos_y_categoria] uvicorn app:main --reload
>> raise PydanticSchemaGenerationError(
...7 lines...
)
pydantic.errors.PydanticSchemaGenerationError: unable to generate pydantic-core schema for chull-in function app). Set 'arbitrary_types_allowed=True' in the model_config to ignore this error or implement '__get_pydantic_core_schema__' on your type to fully support it.
If you get this error by calling handler(some type) within '__get_pydantic_core_schema__', then you likely need to call 'handler.generate_schema(some type)' since we do not call '__get_pydantic_core_schema__' on 'some type' otherwise to avoid infinite recursion.
For further information visit https://errors.pydantic.dev/2.11/usage/schema-for-unknown-type
WARNING: Hatcherlin detected changes in 'app/schemas.py'. Reloading...
INFO: Started server process [1000]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50002 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50003 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50004 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
```

(CA05- Crear válida)

The screenshot displays a REST client interface for the 'Categories' API. The selected method is POST, and the endpoint is /categories, labeled 'Create Category'. The 'Parameters' section is empty. The 'Request body' is required and set to 'application/json'. The JSON body is:

```
{
  "name": "Laptops",
  "description": "Computadora portátil.",
  "active": false
}
```

The 'Execute' button is highlighted in blue. Below the interface, a terminal window shows the following logs:

```
(.venv) PS C:\Users\Usuario\Desktop\Proyectos_3ATP\test_fastapi_productos_y_categoria> uvicorn app.main:app --reload
>>
pydantic.errors.PydanticSchemaGenerationError: Unable to generate pydantic-core schema for <built-in function any>. Set 'arbitrary_types_allowed=True' in the model_config to ignore this error or implement '__get_pydantic_core_schema__' on your type to fully support it.
If you got this error by calling handler(some_type) within '__get_pydantic_core_schema__', then you likely need to call 'handler.generate_schema(some_type)' since we do not call '__get_pydantic_core_schema__' on 'some_type' otherwise to avoid infinite recursion.
For further information visit https://errors.pydantic.dev/2.11/u/schema-for-unknown-type
WARNING: Watchfiles detected changes in 'app/schemas.py'. Reloading...
INFO: Started server process [8064]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:58662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:58663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:58664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:58763 - "POST /categories HTTP/1.1" 201 Created
```

(CA06- Nombre demasiado corto)

POST /categories Create Category

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "PC",
  "description": "Computadora portátil.",
  "active": false
}
```

Execute

Clear

Responses		
Curl<pre>curl -X 'POST' \ 'http://127.0.0.1:8080/categories' \ -H 'accept: application/json' \ -H 'content-type: application/json' \ -d '{ "name": "PC", "description": "Computadora portátil.", "active": false }'</pre>		
Request URL http://127.0.0.1:8080/categories		
Server response		
Code	Details	
422	Error: Unprocessable Content Response body<pre>{ "detail": [{ "type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 3 characters", "input": "PC", "ctx": { "min_length": 3 } }] }</pre> Response headers<pre>content-length: 147 content-type: application/json date: Sun, 13 Sep 2026 23:26:45 GMT server: uvicorn</pre>	Download
Responses		
Code	Description	Links
201	Successful Response	No links

(CA07- Falta nombre)

POST /categories Create Category

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "",
  "description": "Computadora portátil.",
  "active": false
}
```

Execute

Clear

Responses

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/categories' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "",
    "description": "Computadora portátil.",
    "active": false
  }'
```

Request URL

http://127.0.0.1:8000/categories

Server response

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 3 characters", "input": "", "ctx": { "min_length": 3 } }] }</pre> Download Response headers <pre>content-length: 145 content-type: application/json date: Sun, 13 Sep 2026 23:23:32 GMT server: unicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

(CA08- Actualizar existente)

PATCH /categories/{category_id} Update Category

Parameters

Cancel

Name

Description

category_id required

integer (path)

1

Request body required

application/json

Edit Value | Schema

```
{  "name": "Nombre de la categoria actualizado",  "description": "Descripcion de la categoria actualizado",  "active": true}
```

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \  'http://127.0.0.1:8000/categories/1' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Nombre de la categoria actualizado",  "description": "Descripcion de la categoria actualizado",  "active": true}'
```

Request URL

```
http://127.0.0.1:8000/categories/1
```

Server response

Code

Details

200

Response body

```
{  "name": "Nombre de la categoria actualizado",  "description": "Descripcion de la categoria actualizado",  "active": true,  "id": 1}
```

Response headers

```
content-length: 122  content-type: application/json  date: Sun, 13 Sep 2026 23:26:36 GMT  server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

(CA09- Actualizar inexistente)

PATCH /categories/{category_id} Update Category

Parameters

Cancel

Name	Description
category_id required	0
integer (path)	

Request body required

application/json

Edit Value | Schema

```
{  "name": "Nombre de la categoria actualizado",  "description": "Descripción de la categoria actualizado",  "active": true}
```

Execute

Clear

Responses

Responses

Curl

```
curl -X 'PATCH' \  'http://127.0.0.1:8000/categories/0' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Nombre de la categoria actualizado",  "description": "Descripción de la categoria actualizado",  "active": true}'
```

Request URL

http://127.0.0.1:8000/categories/0

Server response

Code	Details
404	Error: Not Found
Undocumented	Response body <pre>{ "detail": "Category not found"}</pre> Download Response headers <pre>content-length: 31 content-type: application/json date: Sun, 13 Sep 2026 23:28:55 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

(CA10- Eliminar existente)

DELETE /categories/{category_id} Delete Category

Parameters

Cancel

Name	Description
category_id * required	
integer	
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/categories/1' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/categories/1
```

Server response

Code	Details
204	Response headers <pre>date: Sun, 13 Sep 2026 23:30:25 GMT server: uvicorn</pre>

```
(.venv) PS C:\Users\Usuario\Desktop\Proyectos_JATP\test_fastapi_productos_y_categorias> uvicorn app.main:app --reload
>>
WARNING: WatchFiles detected changes in 'app\schemas.py'. Reloading...
INFO: Started server process [8904]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50763 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:50770 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50775 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50799 - "PATCH /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50800 - "GET /categories HTTP/1.1" 200 OK
INFO: 127.0.0.1:50802 - "PATCH /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50805 - "DELETE /categories/1 HTTP/1.1" 204 No Content
[]
```

(CA11- Eliminar inexistente)

DELETE /categories/{category_id} Delete Category

Parameters Cancel

Name	Description
category_id * required integer (path)	0

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  http://127.0.0.1:8000/categories/0' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/categories/0
```

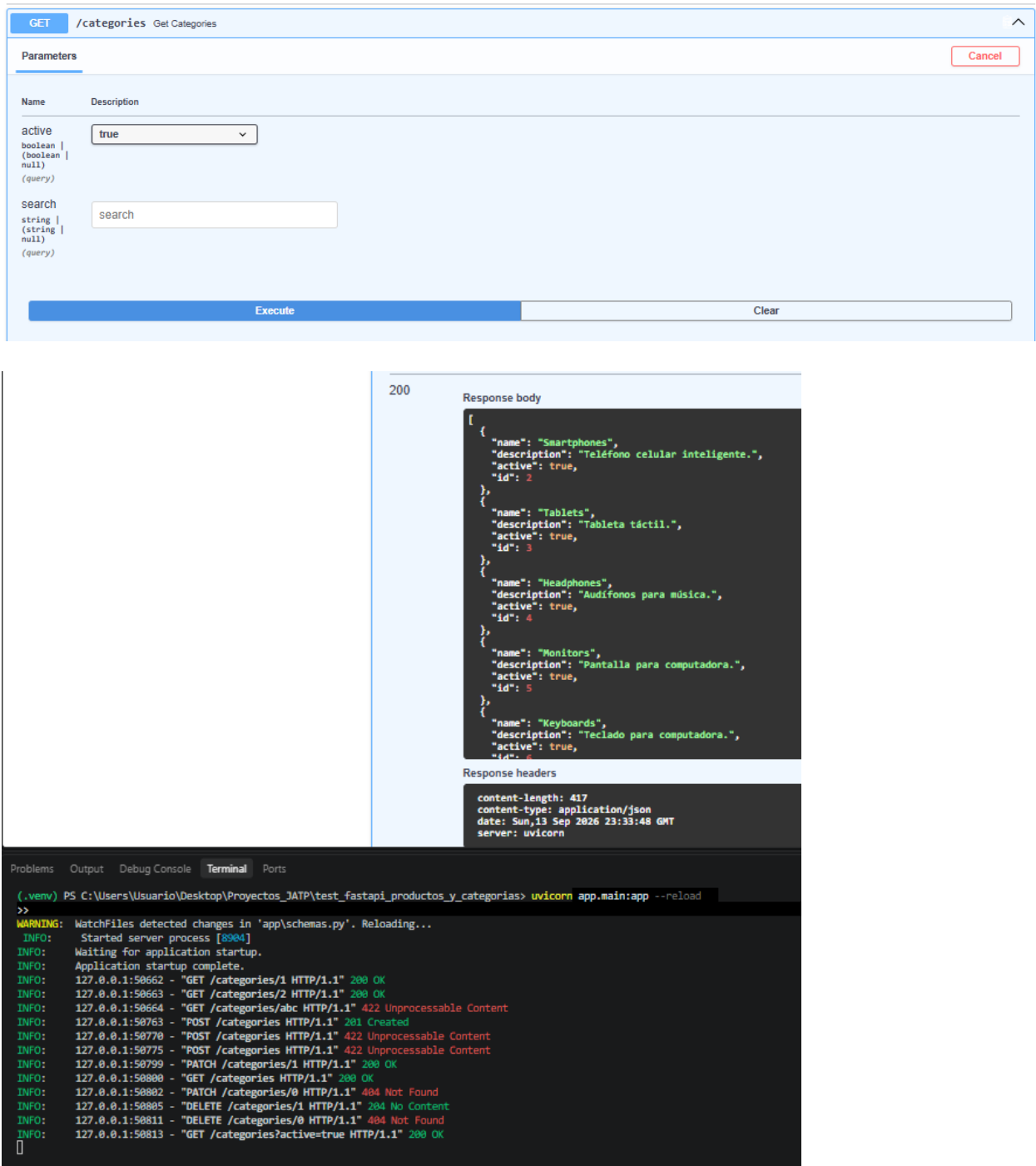
Server response

Code	Details
404 Undocumented	Error: Not Found Response body <pre>{ "detail": "Category not found" }</pre> Download

```
Problems Output Debug Console Terminal Ports

(.venv) PS C:\Users\Usuario\Desktop\Proyectos_JATP\test_fastapi_productos_y_categorias> uvicorn app.main:app --reload
>>
WARNING: WatchFiles detected changes in 'app\schemas.py'. Reloading...
INFO: Started server process [8904]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50763 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:50770 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50775 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50799 - "PATCH /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50800 - "GET /categories HTTP/1.1" 200 OK
INFO: 127.0.0.1:50802 - "PATCH /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50805 - "DELETE /categories/1 HTTP/1.1" 204 No Content
INFO: 127.0.0.1:50811 - "DELETE /categories/0 HTTP/1.1" 404 Not Found
[]
```

(CA12- Filtrar activas)



Matriz de Casos de Prueba (CA01–CA12)

A continuación dejo la matriz de casos de prueba que me pide la guía de la actividad. La columna "Resultado" la lleno con PASS o FAIL según lo que me arrojó pytest -v al correr la suite completa.

ID	Escenario	Petición / condición	Esperado	Resultado (PASS/FAIL)
CA01	Listar categorías	GET /categories	200 y lista JSON	PASS
CA02	Consultar existente	GET /categories/1	200	PASS
CA03	Consultar inexistente	GET /categories/0	404	PASS
CA04	ID inválido	GET /categories/abc	422	PASS
CA05	Crear válida	POST con datos correctos	201	PASS
CA06	Nombre demasiado corto	POST con name de 1-2 caracteres	422	PASS
CA07	Falta nombre	POST sin name	422	PASS
CA08	Actualizar existente	PATCH /categories/1	200	PASS
CA09	Actualizar inexistente	PATCH /categories/0	404	PASS
CA10	Eliminar existente	DELETE /categories/1	204	PASS
CA11	Eliminar inexistente	DELETE /categories/0	404	PASS
CA12	Filtrar activas	GET /categories?active=true	200 y solo active=true	PASS

Lista de Verificación de Evidencias de Entrega

- Repositorio o carpeta del proyecto con el código fuente completo.
- Archivo requirements.txt.
- README.md con instrucciones para instalar, ejecutar la API y correr las pruebas.
- Captura de Swagger (/docs) mostrando todos los endpoints obligatorios.
- Captura o evidencia de la ejecución de pytest -v con la suite completa.

- Matriz de casos de prueba (CA01–CA12) diligenciada con el resultado obtenido: PASS o FAIL.

Observaciones Finales

Cuando corrí todas las pruebas con el comando `pytest -v`, me di cuenta que los doce casos de prueba (del CA01 al CA12) pasaron bien, todos salieron en PASSED y no me dio ningún error. Eso lo pude comprobar tanto en las capturas de Swagger como en la terminal.

Para las pruebas donde tenía que consultar algo que no existe (CA03, CA09 y CA11) use el id 0, porque en mi base de datos que tengo en memoria ese id tampoco existe, entonces me dio el resultado que se esperaba, que era el error 404 diciendo que la categoría no fue encontrada.

También pude hacer el reto opcional de la búsqueda por nombre, o sea el `?search=`, lo probé escribiendo la palabra "Mice" y la API sí me devolvió bien la categoría que tenía esa palabra, sin importar si la escribía con mayúscula o minúscula.

Aparte de eso, en mi proyecto también dejé hechos los endpoints de productos (`/products`) con sus pruebas, aunque esa parte no era obligatoria para esta actividad, pero quise dejarla porque ya la tenía hecha de antes.

En general puedo decir que la API me funcionó bien, cumplió con todos los códigos que pedía la guía (200, 201, 204, 404 y 422), las validaciones de Pydantic funcionaron como debían y las pruebas no dependieron del orden en que las ejecuté.